

Minimax Algorithm for Gaming

Aim

To implement the **Minimax algorithm** for making optimal decisions in a two-player game.

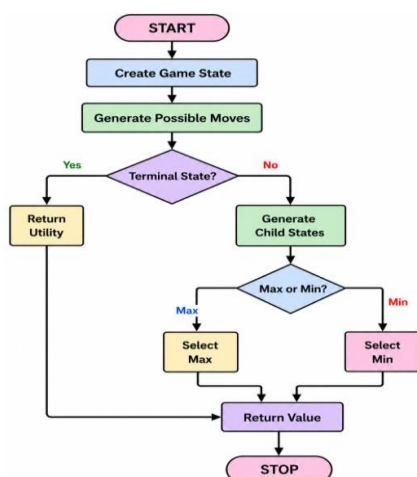
Objectives

- To understand game-tree search.
- To implement the Minimax algorithm.
- To select the best possible move.
- To understand maximizing and minimizing players.

Algorithm

1. Start.
2. Create the game tree.
3. Check whether the current state is terminal.
4. If terminal, return its utility value.
5. For the maximizing player, select the maximum value.
6. For the minimizing player, select the minimum value.
7. Continue recursively.
8. Select the best move.
9. Stop.

Flowchart



PROGRAM

```
def minimax(depth, node, maximizing):
    if depth == 0:
        return node

    if maximizing:
        return max(
            minimax(depth - 1, node + 2, False),
            minimax(depth - 1, node + 3, False)
        )
    else:
        return min(
            minimax(depth - 1, node + 2, True),
            minimax(depth - 1, node + 3, True)
        )

result = minimax(3, 0, True)
print("Best Value:", result)
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/L
ssssssssssssssssssssssssssssssssss
Best Value: 8
```

Result

The Minimax algorithm was successfully implemented to determine the best possible decision for the maximizing player.